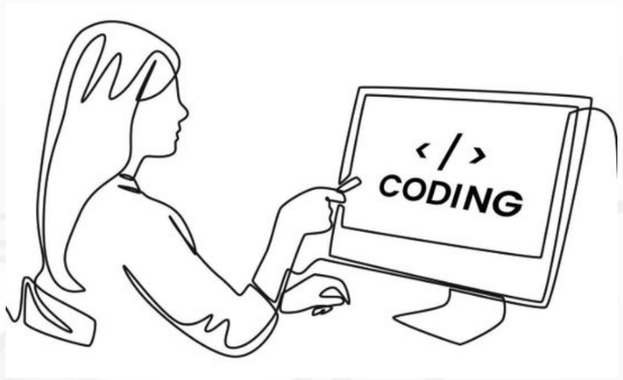
FADENA
FACULTAD DE DEFENSA NACIONAL

Lenguajes de Programación

Codificación en C - Estructuras de control

Docentes:

- Ing. Sebastián Balestrini
- Esp. Ing. Gustavo Di Stefano



Objetivos

- Operadores: Aritmeticos, de Comparación, Lógicos y precedencia
- Uso de estructuras de control en C
- Ciclo For
- Ciclo While
- Ciclo Do - While



Herramientas

OnlineGDB es un entorno de desarrollo integrado (IDE) en la nube que combina edición de código, compilación y ejecución de programas en C/C++ con un solo clic: https://www.onlinegdb.com/

Operadores en C

- Son símbolos especiales que permiten realizar operaciones con datos.
- Tipos principales:
    - Aritméticos
    - Comparación
    - Lógicos
    - Asignación compuestos
    - (Avanzados: bitwise, punteros, estructuras, misceláneos)



FADENA
FACULTAD DE DEFENSA NACIONAL

Operadores aritméticos

Binarios (dos operandos)

Operador	Nombre	Ejemplo
=	asignación	a = 1
+	suma	a + b
-	resta	a - b
*	multiplicación	a * b
/	división	a / b
%	módulo	a % b

Uniarios (un solo operando)

Operador	Nombre	Ejemplo
++	incremento	a++ o ++a
--	decremento	a-- o --a

Diferencia:

- a++ → incrementa después de usar el valor.
- ++a → incrementa antes de usar el valor.

Ejemplo

```
1  int a = 2;
2  int b;
3
4  b = a++; // b es 2, a es 3
5  b = ++a; // b es 4, a es 4
6
```


FADENA
FACULTAD DE DEFENSA NACIONAL

Operadores

Operadores de Comparación

Operador	Nombre	Ejemplo
==	igual a	a == 1
!=	diferente de	a != b
>	mayor que	a > b
<	menor que	a < b
>=	mayor o igual que	a >= b
<=	menor o igual que	a <= b

Operadores Lógicos

Operador	Nombre	Ejemplo
!	NOT	!a
&&	AND	a && b
||	OR	a || b

Ejemplo

```c
if (edad > 18 && edad < 65) {
    printf("Edad laboral\n");
}
```

Precedencia de Operadores

Orden de ejecución en expresiones:

1. Unarios **++ --**
2. Multiplicación / División / Módulo *** / %**
3. Suma / Resta **+ -**
4. Asignación **=**

Asociatividad: es generalmente de izquierda a derecha, excepto en los unarios y en la asignación, que son de derecha a izquierda

Ejemplo

```
1  int a = 2;
2  int b = 4;
3  int c = b + a * a / b - a;
```

```
se evalúa así:

a * a = 4
4 / b = 1
b + 1 - a = 4 + 1 - 2

Resultado final: c = 3
```

Condicionales en C - (if / else)

if / else

- Permiten tomar decisiones según condiciones.
- Ejecuta un bloque de código si la condición es verdadera.
- Opción **else** para el caso contrario.
- Error común: confundir = (asignación) con == (comparación).

a = 5; → asigna el valor 5 a a.

a == 5; → compara si a es igual a 5.

Ejemplo

```
1 ▾ if (condición) {
2       // bloque si es verdadero
3 ▾ } else {
4       // bloque si es falso
5   }
6
```

Condicionales en C - (switch)

switch

- Se usa cuando una variable debe evaluarse contra múltiples valores posibles.
- Más claro y ordenado que muchos if/else if.
- break evita el “efecto cascada” (ejecutar más de un bloque).
- default asegura cubrir cualquier otro valor.
- Muy útil con menús y opciones de usuario.

Ejemplo

```c
switch (variable) {
    case 1:
        // bloque si variable == 1
        break;
    case 2:
        // bloque si variable == 2
        break;
    default:
        // bloque si ningún caso coincide
}
```

Estructuras Repetitivas - for

for - Estructura repetitiva exacta

- Se usa cuando sabemos cuántas veces se repetirá el bucle.
- Toda la configuración está en la cabecera:
  - **Inicialización** → valor inicial de la variable.
  - **Condición** → controla cuántas veces se repite.
  - **Incremento/Decremento** → modifica la variable de control.

Ejemplo

```
for (int i = 0; i < 5; i++) {
    printf("%d\n", i);
}
```

Estructuras Repetitivas - inexactas

while - Estructura repetitiva inexacta (0 a n)

- La condición se evalúa antes de entrar al bucle.
- Ejecuta el bloque mientras la condición sea verdadera.
- Puede ejecutarse 0 veces si la condición inicial es falsa.

```c
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

do while - Estructura repetitiva inexacta (1 a n)

- La condición se evalúa al final del ciclo.
- Variante del while.
- El bloque se ejecuta al menos una vez, incluso si la condición es falsa desde el inicio.

```c
int i = 0;
do {
    printf("%d\n", i);
    i++;
} while (i < 5);
```

Que aprendimos hasta acá ?

- Operadores:
  - Aritmeticos
  - de Comparación
  - Lógicos
  - Precedencia de operadores
- Uso de estructuras de control en C
- Ciclo For
- Ciclo While
- Ciclo Do - While



Próximo paso

Algoritmos básicos, vectores y matrices.

¿Preguntas o dudas?



FADENA
FACULTAD DE DEFENSA NACIONAL